Laboratory work
40% of grade

Requirements

- **Each laboratory work should be presented at the laboratory and uploaded into the corresponding assignments. A laboratory work that is just uploaded without being presented at the lab is not considered to be delivered (no point will be received).**

- **Each laboratory work contains:**
  - **Implementation (source codes)**
  - **Documentation:**
    - **Design specification:** provide all the design decision that you made: partitioning, communication etc
    - **Theoretical performance analysis:** Time complexity, Speed-up, Efficiency, and Cost.
    - **Implementation specification:** provide all details about the optimization that you apply at the implementation level.
    - **Testing –** empirical analysis based on concrete measurements of the execution times**.**
- **If you need to upload an archive then, assure the fact that only the source codes are archived, and the documentation file is uploaded separately!**

- **Use the same files for testing for all the experiments. Do not generate each time new files.**
- **Laboratories could not be delivered in the re-examination week, they should be delivered before the exams' session!**

Problem:
Compute the product of two matrices – of size MxM.

C= AxB

Input: *M (uint32), FileA (string), FileB(string), FileC(string)*
PreCond:
FileA contains the values of A(MxM) -- (real values ->double)

FileB contains the values of B(MxM) -- (double)

Output: ***FileC,*** *t_reading, t_addition, t_writing, t_total*
PostCond:
FileC contains the values of C(MxM) -- (double)
t_reading = execution time needed for reading the matrices
t_addition= execution time for multiplying the matrices
t_writing = execution time for writing matrix C
t_total = total execution time

Implementation language: C++
All the tests should be done on the cluster Kotys.

Variants:

1.Sequential

- sequential reading
- sequential multiplication
- Sequential writing

Theoretical performance analysis: Time complexity

a) text files

b) **binary files** (binary representation of the real numbers)

For the following experiments all the files have binary type

- **Parallel reading is possible if all elements are stored using an equal number of bytes; the file_pointer will be set differently for each thread/process**

2.Paralel - explicit multithreading:

a)

- sequential reading
- parallel multiplication
- Sequential writing

No_Threads = 20,40

b)

- parallel reading
- parallel multiplication
- Sequential writing

No_Threads = 20,40

3.Parallel - OpenMP:

a)

- sequential reading
- parallel multiplication
- Sequential writing

No_Threads = 20, 40

b)

- parallel reading
- parallel multiplication
- sequential writing

Theoretical performance analysis: Time complexity, Speed-up, Efficiency, and Cost

4.Parallel - MPI: (4 cluster nodes)

<u>Cannon algorithm</u> – using groups and communicators

(be careful regarding the Cannon algorithm – implementation should strictly follow this algorithm)

a)

- sequential reading
- parallel multiplication
- Sequential writing

No_Processes = 25(5x5), 64(8x8)

b)

- parallel reading
- parallel multiplication
- sequential writing

No_Processes = 25(5x5), 64(8x8)

5.Parallel - MPI+Multithreading: (4 cluster nodes)

a)

- sequential reading
- parallel multiplication
- Sequential writing

No_Processes = 25(5x5), 64(8x8)

No_Threads = 10 for each process

b)

- parallel reading
- parallel multiplication
-
- sequential writing

No_Processes = 25(5x5), 64(8x8)
No_Threads = 10

6.Parallel -CUDA

- sequential reading
- parallel multiplication
- Sequential writing

Size of Blocks = 1024, 2048 – to be analyzed

7.Parallel - MPI+CUDA: (4 cluster nodes)

a)

- sequential reading
- parallel multiplication
- Sequential writing

Size of Blocks = 1024, 2048
No_Processes = 4

b)

- parallel reading
- parallel multiplication
- sequential writing

Size of Blocks = 1024, 2048
No_Processes = 4

Test cases:

1) M=1000
2) M=100000

10 executions => average corresponding times

For each test case, a table with execution times for

T_reading	T_addition	T_writing	T_total

Finally for both test cases (m=1000, M=100000) two tables should be provided. They are constructed iteratively by adding values for each variant.

Var	T_reading	T_addition	T_writing	T_total	Speed-up (T_seq/T_par)
1.a					
1.b					
2.q					
2.b					
3.a					
3.b					
4.a					
4.b					
5.a					
5.b					
6					
7.a					
7.b					

Deadlines:

D1(week 3+week4) ; D2(week 5+week6);D3(week7+week8); D4(week9+week10); D5(week11+week12);

D1: 1a+1b (sequential) - reading writing from/to files (text files and binary files)
D2: 2.a+2.b si 3.a+3.b (multithreading, OpenMP)
D3: 4.a+4.b si 5.a+5.b (MPI , MPI hybrid)
D4: 6 si 7.a+7.b (CUDA , MPI+CUDA)

Delays (-2points) for each deadline